Tree

沈威宇

November 11, 2026

1 Tree

I Tree

i Tree

A tree is an abstract data type of a set of nodes. A relationship between two nodes i, j is defined with

- i is called a child of j and
- j is called a parent of i; and
- a node can have at most one parent.

If not empty, a tree is recursively defined as a distinguished node r called root and zero or more nonempty trees called subtrees, where the root of each substree is a child of r.

ii Terminology

- Siblings: Nodes sharing the same parent are called siblings. Children of a node are defined with a left to right order.
- Leaf: A node with no children is called a leaf, external node, or exterior node.
- Internal node: A node that is not a leaf is called an internal node or interior node.
- Depth: The number of edges from the root to a node is called the depth of the node, while the root has depth 0.
- Height: The height of a leaf is defined as 0. The height of an internal node is defined as the maximum height of its children plus 1. The maximum depth of the nodes, i.e., the height of its root, is called the height or depth of the tree and denoted h. The height or depth of an empty tree is defined as -1. Some define heights of nodes and trees as the definition here plus 1, which is not used here.
- Ancestors: The ancestors of a node is recursively defined as itself for root and itself and the ancestors of its parent for non-root nodes.
- Descendants: The descendants of a node is recursively defined as itself for leaves and itself and the descendants of its children for internal nodes.
- Level: Nodes of the same depth are called a level. Levels are ordered by depth. The left-to-right order of nodes in a level is defined recursively to, if they are siblings, follow that of siblings; otherwise follow that of their parents.
- Subtree: A tree consisting of all descendants of a node is called a subtree. The left-to-right order of subtrees whose roots are of the same depth are defined as

that of their roots. A subtree whose root is a child of a node is called a subtree of the node.

- The umber of nodes in a tree is denoted n.

iii Tree representation

A representation of tree where each pair of parent and child is connected with a edge.

iv Left child, right sibling representation

A representation of tree where each internal node and its leftmost child is connected with a edge, and each sibling that is not the leftmost one is called to the sibling at its left.

v Skewed

A tree whose every level contains only one node is called skewed.

II n-ary tree

i n-ary tree

A tree in which no node can have more than n children is called an n-ary tree. A child is indexed into one and only one integer in $[1, n]$ even if its parent has less than n children.

ii Full, proper, or plane

An n-ary tree where each internal node has exactly n children is called full, proper, or plane.

iii Filled

A level of depth $k \in \mathbb{N}$ of an n-ary tree is called filled iff each node in the level of depth $k - 1$ has exactly n children. The level of depth 0 is called filled iff the tree is nonempty.

iv Complete

An n-ary tree where every level, except possibly the last, is filled and whose nodes in the last level are as far left as possible, is called complete.

A complete n-ary tree is indexed from top to bottom and then from left to right, with the root as node 1.

v Perfect

A complete n-ary tree where all leaves are of the same depth is called perfect.

A perfect n-ary tree with height k has $\sum_{j=0}^{k} n^j$ nodes.

III Binary tree

i Binary tree

In a binary tree, the two possible children of a node are called left child and right child. A child is either left or right but not both even if it has no sibling. The left and right subtrees of a node x are denoted left(x) and right(x) respectively.

ii Complete binary tree

For a complete binary tree with n nodes:

- Parent of node i is node $\left\lfloor \frac{i}{2} \right\rfloor$ for all integers $i > 1$.
- The left child of node i is $2i + 1$ for all integers $i \leq \frac{n-1}{2}$.
- The right child of node i is $2i$ for all integers $i \leq \frac{n}{2}$.

iii Perfect binary tree

A perfect binary tree with height k has $2^{k+1} - 1$ nodes.

iv Balanced tree

A type of binary tree with height $O(\log n)$ where n is the number of nodes is called balanced.

v Balance factor (bf)

The height of the left subtree minus the height of the right subtree of a node in a binary tree is called the balance factor of the node.

vi Left-skewed and right-skewed

A binary tree whose every internal node has left child and does not have right child is called left-skewed.

A binary tree whose every internal node has right child and does not have left child is called right-skewed.

IV Tree traversal

They are defined recursively. Suppose the root is r.

i Preorder traversal

First visit r, and then visit the subtrees of r from left to right.

ii Postorder traversal

Visit the subtrees of r from left to right, and finally visit r.

iii Inorder traversal (binary tree)

First visit the left subtree of r, and then visit r, and then visit the right subtree of r.

iv Full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals

A full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a full binary tree from:
 *   P - preorder array (size num)
 *   Q - postorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *Q, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  Node *root = createNode(P[0]);
  int leftRootVal = P[1];
  int k = 0;
  while (Q[k] != leftRootVal)
```

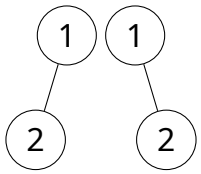
```
    k++;
  int leftSize = k + 1;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, Q, leftSize);
  root->right = construct(P + 1 + leftSize, Q + leftSize, rightSize);
  return root;
}
```

□

v Binary tree with distinct values cannot be uniquely determined by its preorder and postorder traversals

The statement that a binary tree with distinct values can be uniquely determined by its preorder and postorder traversals is false.

Proof. No. A counterexample is the following two binary trees



Both trees have preorder traversal 1, 2, and postorder traversal 2, 1, but they are different. □

vi Binary tree with distinct values can be uniquely determined by its preorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its preorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
```

```
 *   P - preorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  int rootVal = P[0];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, I, leftSize);
  root->right = construct(P + 1 + leftSize, I + leftSize + 1, rightSize);
  return root;
}
```

□

vii Binary tree with distinct values can be uniquely determined by its postorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its postorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
 *   Q - postorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *Q, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(Q[num - 1]);
```

```
  int rootVal = Q[num - 1];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - k;
  root->left = construct(I, Q, leftSize);
  root->right = construct(I + k + 1, Q + leftSize, rightSize);
  return root;
}
```

□

V Binary search tree (BST)

i Binary search tree (BST)

A binary search tree stores keys in the nodes of a binary tree so that searching, insertion and deletion can be done efficiently. For every node x, all the keys in its left subtree are smaller than x and all keys in its right subtree are larger than x. Thus, the values are distinct.

ii Searching

Suppose we are searching for x and the root is r.

- If $x = r$, then we are done.
- If $x < r$, then we search in the left subtree.
- If $x > r$, then we search in the right subtree.

The time complexity is $O(h)$.

iii Finding minimum and maximum

To find the node containing the smallest key, start at root and go to the left child as long as there is one, and the leaf we stop at is the smallest key. Similarly for find the largest key.

The time complexity is $O(h)$.

iv Insertion

Suppose we want to insert x. First search for it, if we found it, do noting (or update something); otherwise, if we stop at y,

- if $x > y$, insert x as the right child of y if $x > y$, and
- if $x < y$, insert x as the left child of y.

The time complexity is $O(h)$.

v Deletion

Suppose we want to delete x. First search for it, if we don't found it, do noting (or update something); otherwise,

- if it is a leaf, delete it,
- if it has one child y, adjust y as a child of the parent of x of the same position as x and delete x, and
- if it has two children, choose either of the policies:
  - inorder successor: replace the value of x with the value of the minimum element y in the right subtree of x, i.e. inorder successor, i.e., the leftmost descendant of the right child of x, which has no left child but may have right child, and
    - remove node y if it has no child, and
    - replace node y with its right child if it has one;
  - inorder predecessor: replace the value of x with the value of the maximum element y in the left subtree of x, i.e. inorder predecessor, i.e., the rightmost descendant of the left child of x, which has no right child but may have left child, and
    - remove node y if it has no child, and
    - replace node y with its left child if it has one;

The time complexity is $O(h)$.

vi (Simple) rotation

A (simple) rotation on an edge between a node x and its parent p, also called rooted at or on p, with p called root and x called pivot of the rotation, is a tree operation on BST and has the following two symmetric variants:

- Right rotation: This is done when x is a left child of p. We perform the following assignments:

$$\text{left}(p) = \text{right}(x)$$

$$\text{right}(x) = p$$

- Left rotation: This is done when x is a right child of p. We perform the following assignments:

$$\text{right}(p) = \text{left}(x)$$

$$\text{left}(x) = p$$

vii Uniqueness given preorder traversal

A BST is uniquely determined by its preorder traversal.

Proof. A BST with one node is obviously unique. The first element must be root, the second element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

viii Uniqueness given postorder traversal

A BST is uniquely determined by its postorder traversal.

Proof. A BST with one node is obviously unique. The last element must be root, the first element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

ix Binary tree is BST iff inorder traversal is in ascending order

A binary tree with distinct values is a binary search tree iff its inorder traversal is sorted in ascending order.

Proof. Only if: Prove by mathematical induction. The inorder traversal of a BST with one node is sorted in ascending order. Assume the inorder traversal of any BST with fewer than n nodes of distinct values is sorted in ascending order. Consider any BST with n nodes, the inorder traversal visits the left subtree, which is smaller than root and itself sorted in ascending order since it's a BST with $n - 1$ nodes, and then root, and then the right subtree, which is larger than root and itself sorted in ascending order since it's a BST with $n - 1$ nodes. Thus, we conclude that the inorder traversal of any BST with n nodes is sorted in ascending order. By mathematical induction, the statement holds.

If: Prove by mathematical induction. A binary tree with one node is a BST. Assume that any binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order is a BST. Consider any binary tree with n nodes of distinct values and root r whose inorder traversal is sorted in ascending order. The subsequence before r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are smaller than r. The subsequence after r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are larger than r. Since the left subtree is a BST and smaller than root and the right subtree is a BST and larger than root, we conclude that any binary tree with n nodes of distinct values whose

inorder traversal is sorted in ascending order is a BST. By mathematical induction, the statement holds. □

x Number of distinct BST shapes

The number T_n of distinct shapes of BSTs with n nodes is:

$$T_0 = 1$$

$$T_n = \sum_{i=0}^{n-1} T_i T_{n-1-i}, \quad n \in \mathbb{N}$$

$$T_n = \frac{1}{(n+1)} \binom{2n}{n}, \quad n \in \mathbb{N}_0$$

Proof. Obviously

$$T_0 = T_1 = 1$$

Let the values be

$$k_1 < k_2 < \ldots < k_n$$

Suppose we choose k_i as root, where $1 \leq i \leq n$. The left subtree contains $k_1, \ldots, k_{i-1}$, so it has $i-1$ nodes and T_{i-1} possible combinations. The right subtree contains $k_{i+1}, \ldots, k_n$, so it has $n-1$ nodes and T_{n-i} possible combinations.

Summing over all choices gives

$$T_n = \sum_{i=1}^{n} T_{i-1} T_{n-i} = \sum_{i=0}^{n-1} T_i T_{n-1-i}$$

Using generating function:

$$C(x) = \sum_{n=0}^{\infty} T_n x^n$$

$$C(x) = 1 + xC(x)^2$$

$$C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}$$

$$C(0) = \mp \frac{1}{\sqrt{1-4x}} = T(0) = 1$$

$$C(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

$$
\begin{aligned}
\sqrt{1-4x} &= \sum_{k=0}^{\infty} \binom{0.5}{k} (-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{\prod_{i=0}^{k-1}(0.5-i)}{k!}(-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{2^k \prod_{i=0}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{2k-2} i}{k! \prod_{i=1}^{k-1} 2i}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k (2k-2)!}{k! 2^{k-1}(k-1)!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2(2k-2)!}{k!(k-1)!}x^k
\end{aligned}
$$

$$
\begin{aligned}
C(x) &= \frac{1-\sqrt{1-4x}}{2x} \\
&= \sum_{k=1}^{\infty} \frac{(2k-2)!}{k!(k-1)!}x^{k-1} \\
&= \sum_{n=0}^{\infty} \frac{(2n)!}{(n+1)!n!}x^n \\
&= \sum_{n=0}^{\infty} \frac{1}{(n+1)}\binom{2n}{n}x^n
\end{aligned}
$$

$$
T_n = \frac{1}{(n+1)}\binom{2n}{n}, \quad n \in \mathbb{N}_0
$$

□

VI AVL tree (Adelson-Velsky and Landis tree)

i AVL tree (Adelson-Velsky and Landis tree)

An AVL tree is a self-balancing binary search tree (BST) that keeps info of height, invented by Georgy Adelson-Velsky and Evgenii Landis. An AVL tree needs to comply with the property that for every node with two children in the tree, the height of its left and right subtrees differ by at most 1.

Some define heights of nodes and AVL trees as the definition used here plus 1, which is not used here.

ii Relation of height and numbers of nodes

Let the minimum number of nodes in an AVL tree of height h be N_h. Since the minimum numbers of subtrees height of the root of an AVL tree of height h are $h-1$ and $h-2$, we have:

$$N_{-1} = 0$$

$$N_0 = 1$$

$$N_h = N_{h-1} + N_{h-2} + 1, \quad h \in \mathbb{N}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1, \quad h+1 \in \mathbb{N}_0$$

Proof. Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2 + \sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{7+3\sqrt{5}}{2} = \varphi^4 = 3\varphi + 2$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$\frac{3-\sqrt{5}}{2} = \frac{1}{\varphi^2} = 1 - \frac{1}{\varphi} = 2 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$N_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$N_0 = a + b + c = 1$$

$$N_1 = a\varphi - \frac{b}{\varphi} + c = 0 + 1 + 1 = 2$$

$$N_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 1 + 2 + 1 = 4$$

$$N_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 4$$

$$N_2 - N_1 = a + b = 4 - 2 = 2$$

$$c = N_0 - (a + b) = 1 - 2 = -1$$

$$N_1 - c = a\varphi - \frac{2-a}{\varphi} = 2 + 1 = 3$$

$$a\left(\varphi^2 + 1\right) = 3\varphi + 2$$

$$a = \frac{3\varphi + 2}{\varphi + 2} = \frac{\varphi^4}{\varphi + 2}$$

$$b = 2 - a = \frac{2-\varphi}{\varphi+2} = \frac{1}{\varphi^2(\varphi+2)}$$

$$N_h = \frac{1}{\varphi+2}\left(\varphi^{h+4} + \left(-\frac{1}{\varphi}\right)^{h+2}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1$$

□

So

$$N_h = \Theta(\left(\frac{1+\sqrt{5}}{2}\right)^h)$$

$$h = \log_{\frac{1+\sqrt{5}}{2}} N_h + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Let the maximum number of nodes in an AVL tree of height h be M_h. Since the maximum numbers of subtrees height of the root of an AVL tree of height h are $h-1$, we have:

$$M_{-1} = 0$$

$$M_h = 2M_{h-1} + 1, \quad h \in \mathbb{N}_0$$

$$M_h = 2^{h+1} - 1, \quad h + 1 \in \mathbb{N}_0$$

So

$$M_h \in \Theta(\log_2 M_h)$$

$$h = \log_2 M_h + O(1)$$

Let the number of nodes be n.

$$N_h \le n \le M_h$$

$$\log_2 n + O(1) \le h \le \log_{\frac{1+\sqrt{5}}{2}} n + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Thus

$$h \in \Theta(\log n)$$

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in AVL tree is the same as in BST.

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

iv Rotation

When insertion/deletion, the AVL property may be violated, called unbalanced. Since an insertion/deletion involves adding/deleting a single node, this can only increase/decrease the height of some subtree by 1. Thus, if the AVL property is violated at a node x, it means that the heights of left and right subtrees of it differ by exactly 2. We need to transform the tree to restore the AVL property, called rebalancing, using a single/simple rotation or a double rotation, each has two symmetric variants:

- LL case / right single rotation: Let the left child of x be y. The LL case is where the left subtree of y has height $h+1$, while the right subtree of y and the right subtree of x have height h. We perform a right rotation on the edge between y and x, i.e.,

$$\text{left}(x) = \text{right}(y)$$
$$\text{right}(y) = x$$

- RR case / left single rotation: Let the right child of x be y. The RR case is where the right tree of y has height $h+1$, while the left subtree of y and the left subtree of x have height h. We perform a left rotation on the edge between y and x, i.e.,

$$\text{right}(x) = \text{left}(y)$$
$$\text{left}(y) = x$$

- LR case / left-then-right double rotation: Let the left child of x be y and the right child of y be z. The LR case is where the right subtree of y has height $h+1$, while the left subtree of y and the right subtree of x have height h. We perform a left rotation on the edge between z and y and then a right rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{right}(y) = \text{left}(z)$$
$$\text{left}(z) = y$$
$$\text{left}(x) = \text{right}(z)$$
$$\text{right}(z) = x$$

- RL case / right-then-left double rotation: Let the right child of x be y and the left child of y be z. The RL case is where the left subtree of y has height $h+1$, while the right subtree of y and the left subtree of x have height h. We perform a right rotation on the edge between z and y and then a left rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{left}(y) = \text{right}(z)$$
$$\text{right}(z) = y$$
$$\text{right}(x) = \text{left}(z)$$
$$\text{left}(z) = x$$

v Insertion

Insertion in AVL tree involves first doing insertion as with a normal BST, and then tracing the path from the new leaf to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then break the loop since no second violation may exist.

Proof. Let x be the first node encountered when traversing upward from the newly inserted node toward the root that violates the AVL property. Since an insertion involves adding a single node, this can only increase the height of some subtree by 1. Thus, the heights of the left and right subtrees differ by exactly 2. For any of the four cases (LL, RR, LR, RL), since the inserted node must be in the subtree of height $h+1$ of y, the height of the subtree whose root is x has height $h+2$ both before insertion and after rotation. Thus, the fact that the AVL property is satisfied for all ancestors of x before insertion implies that the AVL property is satisfied for all ancestors of x after rotation. □

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

vi Deletion

Deletion in AVL tree involves first doing deletion the same as in BST, and then tracing the path from

- the original parent of the deleted node if the deleted node had zero or one child, and
- the original parent of the inorder successor or inorder predecessor that was moved if the deleted node had two children,

to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then proceed to the parent of x since violation may propagate upwards.

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

vii Number of distinct AVL tree shapes

The number $A_{h,n}$ of distinct shapes of AVL trees with n nodes and height h is:

$$A_{-1,0} = 1$$

$$A_{-1,n} = 0, \quad n \neq 0$$

$$A_{0,1} = 1$$

$$A_{0,n} = 0, \quad n \neq 1$$

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}, \quad h \in \mathbb{N}, n \in \mathbb{N}_0$$

Proof. The root uses one node, leaving $n-1$ nodes for the two subtrees. Since the whole tree has height h, the two trees must either have heights $h-1, h-2$ or both have height $h-1$. Thus,

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}$$

□

viii Number of distinct AVL tree shapes with minimum number of nodes given height

The number B_h of distinct shapes of AVL trees with height h and minimum number of nodes is:

$$B_{-1} = 1$$

$$B_0 = 1$$

$$B_h = 2B_{h-1}B_{h-2}, \quad h \in \mathbb{N}$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right)-1}, \quad h+1 \in \mathbb{N}_0$$

Proof. To minimize the number of nodes, the two subtrees of root must be the AVL tree with height $h-1, h-2$ and minimum number of nodes, which have B_{h-1}, B_{h-2} distinct shapes respectively. There are two possible arrangements:

- Left subtree has height $h-1$, and right subtree has height $h-2$.
- Left subtree has height $h-2$, and right subtree has height $h-1$.

Thus,

$$B_h = 2B_{h-1}B_{h-2}$$

Let

$$C_h = \log_2 B_h$$

$$C_h = C_{h-1} + C_{h-2} + 1$$

Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2+\sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$C_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$C_0 = a + b + c = \log_2 1 = 0$$

$$C_1 = a\varphi - \frac{b}{\varphi} + c = \log_2(2 * 1 * 1) = 1$$

$$C_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 0 + 1 + 1 = 2$$

$$C_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 2$$

$$C_2 - C_1 = a + b = 2 - 1 = 1$$

$$c = C_0 - (a + b) = 0 - 1 = -1$$

$$C_1 - c = a\varphi - \frac{1-a}{\varphi} = 1 + 1 = 2$$

$$a\left(\varphi^2 + 1\right) = 2\varphi + 1$$

$$a = \frac{2\varphi + 1}{\varphi + 2} = \frac{\varphi^3}{\varphi + 2}$$

$$b = 1 - a = \frac{1-\varphi}{\varphi+2} = -\frac{1}{\varphi(\varphi+2)}$$

$$C_h = \frac{1}{\varphi+2}\left(\varphi^{h+3} + \left(-\frac{1}{\varphi}\right)^{h+1}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$C_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right) - 1$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right)-1}$$

□

VII Splay tree

i Splay tree

A splay tree is a self-adjusting binary search tree (BST) that keeps no info of height and in which every access operation moves the accessed node to the root through an operation called splaying, invented by Daniel Dominic Sleator and Robert Endre Tarjan. The idea is locality, that is, if an item is accessed, it is likely to be accessed again soon. The worst-case searching, insertion, and deletion time complexities are as with a normal BST, while the amortized time complexities are $O(\log n)$.

ii (Bottom-up) splay

When a node x is accessed, a (bottom-up) splay operation consisting of a sequence of splay steps is performed on x to move it to the root, called splaying x. Let the parent of x be p and the parent of p be g. There are three types of splay steps, each of which has two symmetric variants:

- Zig step / simple rotation: This is done if x is a child of the root. We perform a rotation on the edge between x and root. This only happens as the last step and only happens when x originally has odd depth.
- Zig-zig step: This is done either when x is a left child of p and p is a left child of g, or when x is a right child of p and p is a right child of g. We perform a rotation on the edge between p and g and then a rotation on the edge between x and p.
- Zig-zag step: This is done either when x is a left child of p and p is a right child of g, or when x is a right child of p and p is a left child of g. We perform a rotation on the edge between x and p and then a rotation on the edge between x, now at the original position of p, and g.

A zig step is a 1-level splay step while a zig-zig or zig-zag step is a 2-level splay step.

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in splay tree involves first doing the operation as with a normal BST and then splay the found node if found.

iv Join

Given two BSTs S and T such that all elements of S are smaller than all elements of T, the following steps can be used to join them to a single BST: either

1. Splay the largest item in S. Now this item is in the root r of S and has no right child.
2. Set the right child of the new root r of S to the root of T.

or

1. Splay the smallest item in T. Now this item is in the root r of T and has no left child.
2. Set the left child of the new root r of T to the root of S.

v Split

Given a BST and a value x, splitting the BST at x returns two new BSTs, one containing all elements less than or equal to x, L, and the other containing all elements greater than x, R. This can be done with:

1. Search for x in the tree. If we found it, let y be that node; otherwise let y be the node we stop at.
2. Splay y.
3. Split its right subtree, which is R, from the rest of the tree, which is L.

vi Insertion

To insert a value x into a splay tree, there are two ways: either

1. Insert x as with a normal BST.
2. Splay x.

or

1. Split at x. However, if x is found in the tree, stop after splaying x.
2. Create a new tree with x as root and the two split trees as its subtrees.

vii Deletion

To delete a node x from a splay tree: either

1. Search for x.
2. Splay x.
3. Delete x and join the two subtrees of x.

or

1. Delete x as with a normal BST.
2. Splay the original parent of the deleted x.

viii Amortized analysis

We define size of a node $s(x)$ as the number of nodes in the subtree whose root is node x, rank of a node as

$$r(x) = \log_2 s(x)$$

and potential function of a tree $P(T)$ is

$$P(T) = \sum_{x \in T} r(x)$$

The actual cost C_i of an operation i is defined as the number of rotations in the operation. Let the state before operation i be denoted by the original functions and the state after operation i be denoted with a prime symbol. The amortized cost A_i of an operation i with actual cost C_i that transforms the tree T_{i-1} to T_i is defined as

$$A_i = C_i + \Delta P = C_i + P(T_i) - P(T_{i-1})$$

- Zig step:

$$C_i = 1$$

$$r'(p) \leq r(p)$$

$$\Delta P = r'(x) + r'(p) - r(x) - r(p) \leq r'(x) - r(x)$$

$$A_i \leq 1 + r'(x) - r(x)$$

- Zig-zig step:

$$C_i = 2$$

$$r'(p) \leq r(g)$$

$$r(x) \leq r(p)$$

$$s(x) + s'(g) = s'(x) - 2 \leq s'(x)$$

The log function is concave, i.e.,

$$\frac{\log_2 a + \log_2 b}{2} \leq \log_2\left(\frac{a+b}{2}\right)$$

i.e.,

$$log_2 a + \log_2 b \leq 2\log_2(a+b) - 2$$

Hence

$$r(x) + r'(g) = 2r'(x) - 2$$

$$r'(g) \leq 2r'(x) - r(x) - 2$$

$$\Delta P = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq r'(x) + r'(g) - 2r(x) \leq 3(r'(x) - r(x)) - 2$$

$$A_i \leq 3(r'(x) - r(x))$$

- Zig-zag step:

$$C_i = 2$$

$$s'(p) + s'(g) = s'(x) - 1 \leq s'(x)$$

The log function is concave, i.e.,

$$\frac{\log_2 a + \log_2 b}{2} \leq \log_2\left(\frac{a+b}{2}\right)$$

i.e.,

$$log_2 a + \log_2 b \leq 2\log_2(a+b) - 2$$

Hence

$$r'(p) + r'(g) \leq 2r'(x) - 2$$

$$r(x) \leq r(p) \leq r(g)$$

$$\Delta P = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq 3(r'(x) - r(x)) - 2$$

$$A_i \leq 3(r'(x) - r(x))$$

When splaying a node x, zero or more 2-level splay steps followed by zero or one 1-level splay step is conducted. Let $r_2(x)$ be the rank of x just after the last 2-level splay step and $r_f(x)$ be the final rank of x. The amortized cost A of splaying x is

$$\begin{aligned} A &\leq 1 + r_f(x) - r_2(x) + 3(r_2(x) - r(x)) \\ &\leq 1 + 3(r_f(x) - r_2(x)) + 3(r_2(x) - r(x)) \\ &= 1 + 3(r_f(x) - r(x)) \\ &\leq 1 + 3(\log_2 n - r(x)) \\ &= O(\log n) \end{aligned}$$

For a sequence of m accesses in a splay tree T, we can bound the total actual cost with

$$\sum_{i=1}^{m} C_i = P(T_0) - P(T_m) + \sum_{i=1}^{m} A_i$$

For every node x,

$$1 \leq s(x) \leq n$$

$$0 \leq r(x) \leq \log_2 n$$

Thus the change of rank $\Delta r(x)$ of a node x is

$$\Delta r(x) \leq \log_2 n$$

Thus the change of potential function is

$$P(T_0) - P(T_m) \leq \sum_{x \in T} \log_2 n = O\left(n \log n\right)$$

ix Weighted analysis

The above analysis can be generalized by assigning each node x a weight $w(x) \geq 1$ and defining $s(x)$ as the sum of weights of nodes in the subtree whose root is node x, called weighted analysis. Note that the previous analysis corresponds to the special case $w(x) = 1$. The sum of all weights is denoted W. The same analysis gives:

$$A \leq 1 + 3(\log_2 n - r(x)) = O(\log\left(\frac{W}{w(x)}\right) + 1)$$

For a sequence of m accesses in a splay tree T, we can bound the total actual cost with

$$\sum_{i=1}^{m} C_i = P(T_0) - P(T_m) + \sum_{i=1}^{m} A_i$$

For every node x,

$$w(x) \leq s(x) \leq W$$

$$\log_2(w(x)) \leq r(x) \leq \log_2 W$$

Thus the change of rank $\Delta r(x)$ of a node x is

$$\Delta r(x) \leq \log_2\left(\frac{W}{w(x)}\right)$$

Thus the change of potential function is

$$P(T_0) - P(T_m) \leq \sum_{x \in T} \log_2\left(\frac{W}{w(x)}\right) = O\left(\sum_{x \in T} \log\left(\frac{W}{w(x)}\right)\right)$$

x Balance theorem

The cost of performing a sequence of m accesses in a splay tree containing n nodes is

$$O(m \log n + n \log n)$$

Proof. Take a constant weight, e.g.,

$$w(x) = 1$$

Then

$$W = n$$

$$A_i = O(\log n)$$

$$P(T_0) - P(T_m) = O(n \log n)$$

□

This theorem implies that splay trees perform at least as well as AVL trees on sequences of at least n accesses.

xi Static optimality theorem

The cost of performing a sequence of m accesses in a splay tree T containing n nodes, with each node accessed at least once and q_x being the number of times node x is accessed, is

$$O\left(m + \sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right)\right)$$

Proof. Let

$$w(x) = q_x$$

Then

$$W = m$$

$$A_i = O\left(\log\left(\frac{m}{q_x}\right) + 1\right)$$

$$\sum_{i=1}^{m} A_i = O\left(\sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right) + m\right)$$

$$q_x \geq 1$$

$$P(T_0) - P(T_m) = O\left(\sum_{x \in T} \log\left(\frac{m}{q_x}\right)\right) = O\left(\sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right)\right)$$

□

xii Working set theorem

The cost of performing a sequence S of m accesses in a splay tree T containing n nodes, with $t(x)$ being the number of distinct nodes accessed before the previous time node x was accessed for each $x \in S$, is

$$O\left(m + n \log n + \sum_{x \in S} \log(t(x) + 1)\right)$$

Proof. Let

$$w(x) = (t(x) + 1)^{-2}$$

Then

$$W = O(1)$$

$$A_i = O\left(\log(t(x) + 1) + 1\right)$$

$$\sum_{i=1}^{m} A_i = O\left(\sum_{x \in S} \log(t(x) + 1) + m\right)$$

$$P(T_0) - P(T_m) = O\left(n \log n\right)$$

□

xiii Locality property or strong balance theorem

The cost of performing a sequence of m accesses in a splay tree containing n nodes, with k distinct nodes accessed, is

$$O(m + m \log k + n \log n)$$

Proof. Use working set theorem.

$$t(x) = O(k)$$

$$O\left(m + n \log n + \sum_{x \in S} \log(t(x) + 1)\right) = O(m + n \log n + m \log k)$$

□

xiv Top-down splay

An alternative implementation of splay that is done together with searching. Suppose we want to splay x. We start at the root and maintain two trees which are initially empty, left tree L of nodes smaller than x and right tree R of nodes larger than x, during the traversal. In each step with current node c, there are three cases, each with two symmetric variants, where we will describe the former and the latter is the former with left and right exchanged and L and R exchanged:

- $x <$ left(c): We perform a rotation on the edge between c and left(c) and then a rotation one the edge left(left(c)) and left(c), and then assign left(c) to c.
- $x >$ right(c): We perform a rotation on the edge between c and right(c) and then a rotation one the edge right(right(c)) and right(c), and then assign right(c) to c.
- left$(c) < x < c$: We connect c into R, and then assign left(c) to c.
- right$(c) > x > c$: We connect c into L, and then assign right(c) to c.
- $x = c$, $x < c$ and c has no left child, or $x > c$ and c has no right child: The traversal stops at c. We take c as the new root, and L and R as left and right subtrees of c.

Top-down splay has the same worst-case and amortized searching, insertion, and deletion time complexities as bottom-up splay and $O(1)$ extra space.

VIII An implementation of BST, ASL tree, and splay tree in C++

```
1 #include <algorithm>
2 #include <functional>
3 #include <stdexcept>
4 #include <utility>
5 #include <vector>
6 using namespace std;
7
8 template <class T> struct BST_Node {
9   using value_type = T;
```

```
  T value;
  BST_Node<T> *parent;
  BST_Node<T> *left;
  BST_Node<T> *right;

  BST_Node(const T &value, BST_Node<T> *parent = nullptr)
      : value(value), parent(parent), left(nullptr), right(nullptr) {}
};

template <class T, class Node = BST_Node<T>> class BST {
public:
  using value_type = T;

  Node *root; // Allow root->parent != nullptr

  BST(Node *root = nullptr) : root(root) {}

  BST(const BST &) = delete;

  BST(BST &&other) { root = other.root; }

  BST &operator=(const BST &) = delete;

  BST &operator=(BST &&other) {
    clear();
    root = other.root;
    return *this;
  }

  // 1: left child
  // 0: right child
  // -1: node or its parent is nullptr
  static char is_left_child(Node *node) {
    if (node == nullptr || node->parent == nullptr)
      return -1;
    return char(node->parent->left == node);
  }

  vector<T> preorder(Node *node) const {
    if (node == nullptr)
      return vector<T>();
    vector<T> result;
    vector<Node *> stack;
    stack.push_back(node);
    while (!stack.empty()) {
      Node *node = stack.back();
      stack.pop_back();
      result.push_back(node->value);
      if (node->right != nullptr)
        stack.push_back(node->right);
      if (node->left != nullptr)
        stack.push_back(node->left);
    }
    return result;
  }

  vector<Node *> preorder_node(Node *node) const {
    if (node == nullptr)
      return vector<Node *>();
    vector<Node *> result, stack;
    stack.push_back(node);
    while (!stack.empty()) {
```

```
72        Node *node = stack.back();
73        stack.pop_back();
74        result.push_back(node);
75        if (node->right != nullptr)
76          stack.push_back(node->right);
77        if (node->left != nullptr)
78          stack.push_back(node->left);
79      }
80      return result;
81    }
82
83    // O(n)
84    vector<T> preorder() const { return preorder(root); }
85    vector<Node *> preorder_node() const { return preorder_node(root); }
86
87    vector<T> inorder(Node *node) const {
88      if (node == nullptr)
89        return vector<T>();
90      vector<T> result;
91      vector<Node *> stack;
92      while (!stack.empty() || node != nullptr) {
93        while (node != nullptr) {
94          stack.push_back(node);
95          node = node->left;
96        }
97        node = stack.back();
98        stack.pop_back();
99        result.push_back(node->value);
100       node = node->right;
101     }
102     return result;
103   }
104
105   vector<Node *> inorder_node(Node *node) const {
106     if (node == nullptr)
107       return vector<Node *>();
108     vector<Node *> result, stack;
109     while (!stack.empty() || node != nullptr) {
110       while (node != nullptr) {
111         stack.push_back(node);
112         node = node->left;
113       }
114       node = stack.back();
115       stack.pop_back();
116       result.push_back(node);
117       node = node->right;
118     }
119     return result;
120   }
121
122   // O(n)
123   vector<T> inorder() const { return inorder(root); }
124   vector<Node *> inorder_node() const { return inorder_node(root); }
125
126   vector<T> postorder(Node *node) const {
127     if (node == nullptr)
128       return vector<T>();
129     vector<T> result;
130     vector<Node *> stack;
131     Node *last_visited = nullptr;
132     while (!stack.empty() || node != nullptr) {
133       if (node != nullptr) {
```

```
      stack.push_back(node);
      node = node->left;
    } else {
      Node *right = stack.back()->right;
      if (right != nullptr && last_visited != right) {
        node = right;
      } else {
        result.push_back(stack.back()->value);
        last_visited = stack.back();
        stack.pop_back();
      }
    }
  }
  return result;
}

vector<Node *> postorder_node(Node *node) const {
  if (node == nullptr)
    return vector<Node *>();
  vector<Node *> result, stack;
  Node *last_visited = nullptr;
  while (!stack.empty() || node != nullptr) {
    if (node != nullptr) {
      stack.push_back(node);
      node = node->left;
    } else {
      Node *right = stack.back()->right;
      if (right != nullptr && last_visited != right) {
        node = right;
      } else {
        result.push_back(stack.back());
        last_visited = stack.back();
        stack.pop_back();
      }
    }
  }
  return result;
}

// O(n)
vector<T> postorder() const { return postorder(root); }
vector<Node *> postorder_node() const { return postorder_node(root); }

virtual void clear(Node *node) {
  if (node == nullptr)
    throw invalid_argument("Node is nullptr");
  char iln = is_left_child(node);
  if (iln == 1)
    node->parent->left = nullptr;
  else if (iln == 0)
    node->parent->right = nullptr;
  vector<Node *> stack = preorder_node(node);
  for (auto it = stack.rbegin(); it != stack.rend(); ++it)
    delete *it;
}

// O(n)
virtual void clear() {
  if (root == nullptr)
    return clear(root);
  root = nullptr;
}
```

```
196
197   virtual ~BST() { clear(); }
198
199   // Return node, found
200   // O(h)
201   virtual pair<Node *, bool> search(const T &x) const {
202     if (root == nullptr)
203       return {nullptr, false};
204     Node *node = root;
205     while (true) {
206       if (std::less<>{}(x, node->value)) {
207         if (node->left == nullptr)
208           return {node, false};
209         node = node->left;
210       } else if (std::less<>{}(node->value, x)) {
211         if (node->right == nullptr)
212           return {node, false};
213         node = node->right;
214       } else {
215         return {node, true};
216       }
217     }
218   }
219
220   virtual Node *find_min(Node *node) const {
221     if (node == nullptr)
222       return nullptr;
223     while (node->left != nullptr)
224       node = node->left;
225     return node;
226   }
227
228   // O(h)
229   virtual Node *find_min() const { return find_min(root); }
230
231   virtual Node *find_max(Node *node) const {
232     if (node == nullptr)
233       return nullptr;
234     while (node->right != nullptr)
235       node = node->right;
236     return node;
237   }
238
239   // O(h)
240   virtual Node *find_max() const { return find_max(root); }
241
242   // O(h)
243   Node *inorder_successor(Node *node) const {
244     if (node == nullptr)
245       throw invalid_argument("Node is nullptr");
246     if (node->right != nullptr)
247       return find_min(node->right);
248     Node *p = node->parent;
249     while (p != nullptr && node == p->right) {
250       node = p;
251       p = p->parent;
252     }
253     return p;
254   }
255
256   // O(h)
257   Node *inorder_predecessor(Node *node) const {
```

```
    if (node == nullptr)
      throw invalid_argument("Node is nullptr");
    if (node->left != nullptr)
      return find_max(node->left);
    Node *p = node->parent;
    while (p != nullptr && node == p->left) {
      node = p;
      p = p->parent;
    }
    return p;
  }

  // Return smallest node > x
  // O(h)
  Node *upper_bound(const T &x) const {
    Node *result = nullptr;
    Node *node = root;
    while (node != nullptr) {
      if (std::less<>{}(x, node->value)) {
        result = node;
        node = node->left;
      } else {
        node = node->right;
      }
    }
    return result;
  }

  // Return smallest node >= x
  // O(h)
  Node *lower_bound(const T &x) const {
    Node *result = nullptr;
    Node *node = root;
    while (node != nullptr) {
      if (!std::less<>{}(node->value, x)) {
        result = node;
        node = node->left;
      } else {
        node = node->right;
      }
    }
    return result;
  }

  // Return node, inserted
  // O(h)
  virtual pair<Node *, bool> insert(const T &x) {
    if (root == nullptr) {
      root = new Node(x);
      return {root, true};
    }
    Node *node = root;
    while (true) {
      if (std::less<>{}(x, node->value)) {
        if (node->left == nullptr) {
          node->left = new Node(x, node);
          return {node->left, true};
        }
        node = node->left;
      } else if (std::less<>{}(node->value, x)) {
        if (node->right == nullptr) {
          node->right = new Node(x, node);
```

```
320           return {node->right, true};
321         }
322         node = node->right;
323       } else {
324         return {node, false};
325       }
326     }
327   }
328
329   // O(1)
330   Node *replace(Node *node, Node *replacement) {
331     if (node == nullptr)
332       throw invalid_argument("Node is nullptr");
333     Node *parent = node->parent;
334     if (root == node)
335       root = replacement;
336     if (parent != nullptr) {
337       if (parent->left == node) {
338         parent->left = replacement;
339       } else {
340         parent->right = replacement;
341       }
342     }
343     if (replacement != nullptr)
344       replacement->parent = parent;
345     return parent;
346   }
347
348   // O(1)
349   Node *replace_remove(Node *node, Node *replacement) {
350     Node *result = replace(node, replacement);
351     delete node;
352     return result;
353   }
354
355   // O(1)
356   static void swap_position(Node *a, Node *b) {
357     if (a == nullptr || b == nullptr)
358       throw invalid_argument("Node is nullptr");
359     swap(a->parent, b->parent);
360     swap(a->left, b->left);
361     swap(a->right, b->right);
362     if (a->left != nullptr)
363       a->left->parent = b;
364     if (a->right != nullptr)
365       a->right->parent = b;
366     if (b->left != nullptr)
367       b->left->parent = a;
368     if (b->right != nullptr)
369       b->right->parent = a;
370   }
371
372   // O(h)
373   virtual Node *remove_node(Node *node) {
374     if (node == nullptr)
375       throw invalid_argument("Node is nullptr");
376     if (node->left == nullptr)
377       return replace_delete(node, node->right);
378     if (node->right == nullptr)
379       return replace_delete(node, node->left);
380     Node *succ = inorder_successor(node);
381     swap_position(node, succ);
```

```
382     return replace_delete(node, node->right);
383   }
384
385   // Return parent, removed
386   // O(h)
387   virtual pair<Node *, bool> remove(const T &x) {
388     auto [node, found] = search(x);
389     if (!found)
390       return {node, false};
391     return {remove_node(node), true};
392   }
393
394   Node *rotate_right(Node *node) {
395     if (node == nullptr || node->left == nullptr)
396       throw invalid_argument("Node is nullptr");
397     Node *child = node->left;
398     replace(node, child);
399     node->left = child->right;
400     if (child->right != nullptr)
401       child->right->parent = node;
402     child->right = node;
403     node->parent = child;
404     return child;
405   }
406
407   Node *rotate_left(Node *node) {
408     if (node == nullptr || node->right == nullptr)
409       throw invalid_argument("Node is nullptr");
410     Node *child = node->right;
411     replace(node, child);
412     node->right = child->left;
413     if (child->left != nullptr)
414       child->left->parent = node;
415     child->left = node;
416     node->parent = child;
417     return child;
418   }
419
420   Node *rotate_parent(Node *child) {
421     if (child == nullptr || child->parent == nullptr)
422       throw invalid_argument("Node is nullptr");
423     if (is_left_child(child) == 1)
424       return rotate_right(child->parent);
425     return rotate_left(child->parent);
426   }
427 };
428
429 template <class T> struct AVL_Node {
430   using value_type = T;
431   T value;
432   AVL_Node<T> *parent;
433   AVL_Node<T> *left;
434   AVL_Node<T> *right;
435   unsigned int height; // leaf 1
436
437   AVL_Node(const T &value, AVL_Node<T> *parent = nullptr)
438       : value(value), parent(parent), left(nullptr), right(nullptr), height(1) {
439   }
440 };
441
442 template <class T, class Node = AVL_Node<T>> class AVL : public BST<T, Node> {
443 public:
```

```
444   using Base = BST<T, Node>;
445
446   // O(1)
447   static void swap_position(Node *a, Node *b) {
448     if (a == nullptr || b == nullptr)
449       throw invalid_argument("Node is nullptr");
450     swap(a->parent, b->parent);
451     swap(a->left, b->left);
452     swap(a->right, b->right);
453     swap(a->height, b->height);
454     if (a->left != nullptr)
455       a->left->parent = b;
456     if (a->right != nullptr)
457       a->right->parent = b;
458     if (b->left != nullptr)
459       b->left->parent = a;
460     if (b->right != nullptr)
461       b->right->parent = a;
462   }
463
464   static int height(Node *node) { return node == nullptr ? 0 : node->height; }
465
466   static void update(Node *node) {
467     node->height = 1 + max(height(node->left), height(node->right));
468   }
469
470   static int bf(Node *node) { return height(node->left) - height(node->right); }
471
472   Node *rotate_right(Node *parent) override {
473     Node *result = Base::rotate_right(parent);
474     if (result != nullptr) {
475       update(result->right);
476       update(result);
477     }
478     return result;
479   }
480
481   Node *rotate_left(Node *parent) override {
482     Node *result = Base::rotate_left(parent);
483     if (result != nullptr) {
484       update(result->left);
485       update(result);
486     }
487     return result;
488   }
489
490   Node *rebalance(Node *node) {
491     if (node == nullptr)
492       return nullptr;
493     update(node);
494     int bf = bf(node);
495     if (bf > 1) {
496       Node *child = node->left;
497       if (bf(child) < 0) // LR
498         rotate_left(child);
499       return rotate_right(node); // LL, LR
500     }
501     if (bf < -1) {
502       Node *child = node->right;
503       if (bf(child) > 0) // RL
504         rotate_right(child);
505       return rotate_left(node); // RL, RR
```

```
    }
    if (node->parent == nullptr)
      return node;
    return rebalance(node->parent);
  }

  pair<Node *, bool> insert(const T &x) override {
    auto [node, inserted] = Base::insert(x);
    if (inserted && node->parent != nullptr)
      rebalance(node->parent);
    return {node, inserted};
  }

  pair<Node *, bool> remove(const T &x) override {
    auto [parent, removed] = Base::remove(x);
    if (!removed)
      return {parent, removed};
    Node *node = parent;
    while (node != nullptr) {
      node = rebalance(node)->parent;
    }
    return {parent, removed};
  }
};

template <class T, class Node = BST_Node<T>> class Splay : public BST<T, Node> {
public:
  using Base = BST<T>;

  Node *splay(Node *node) {
    if (node == nullptr)
      return nullptr;
    while (node->parent != nullptr) {
      Node *parent = node->parent;
      Node *grandparent = parent->parent;
      // Zig
      if (grandparent == nullptr) {
        this->rotate(node);
      }
      // Zig-zig
      else if ((parent->left == node && grandparent->left == parent) ||
               (parent->right == node && grandparent->right == parent)) {
        this->rotate(parent);
        this->rotate(node);
      }
      // Zig-zag
      else {
        this->rotate(node);
        this->rotate(node);
      }
    }
    return node;
  }

  pair<Node *, bool> search(const T &x) override {
    auto [node, found] = Base::search(x);
    return {splay(node), found};
  }
};
```